



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК4 Программное обеспечение ЭВМ, информационные технологии

ЛАБОРАТОРНАЯ РАБОТА №6

«Вывод растровых изображений в OpenGL»

по дисциплине: *«Компьютерная графика»*

Выполнил: студент группы ИУК4-43Б

(Подпись)

Афанасьева Ю.Р.

(И.О. Фамилия)

Проверил:

(Подпись)

Тихонович Е.О.

(И.О. Фамилия)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Цель: формирование практических навыков по работе с растровыми изображениями средствами OpenGL, а также простейшим преобразованиям цветовых и пространственных характеристик.

Задачи: понимать принципы вывода растровых изображений, знать отличия битового образа от растрового и их характеристики, научиться использовать средства OpenGL для вывода растровых изображений, знать основные константы OpenGL, используемые при обработке растровых изображений, уметь создавать приложения OpenGL с использованием функций для работы с растровыми изображениями.

Вариант 3

Задание: воспроизвести результаты, представленные в теоретическом обзоре, научиться работать с битовыми масками и пиксельными изображениями, согласно варианту, выполнить преобразование цветового пространства и расположения изображения.

- 1) Для [Листинга 1](#) создать битовые образы согласно варианту
- 2) Для [Листингов 2, 3, 4](#) осуществить вывод изображения в формате tga согласно варианту
- 3) Для [Листинга 5](#) реализовать преобразование изображения согласно варианту.

Этапы выполнения работы:

1. Был создан собственный битовый образ размером 24×24 пикселя. С помощью функции glBitmap 36 копий образа размещены в центре окна в виде квадрата 6×6. Для каждого образа установлен случайный цвет.
2. С помощью функции gltLoadTGA выполнена загрузка собственного изображения формата TGA. Функцией glDrawPixels изображение выведено четыре раза: в исходном виде и отдельно по красной, зелёной и синей компонентам. Изображения расположены от правого верхнего к левому нижнему углу окна.
3. Для исходного TGA-изображения реализовано масштабирование функцией glPixelZoom: по оси X в 2 раза, по оси Y — в 0,7 раза. С использованием карт цветов и функции glPixelTransferi выполнена инверсия цветовых компонентов. Через контекстное меню можно переключаться между исходным и преобразованным изображениями.

Задание 1: Создать свой битовый образ (не из примера) размером 24x24. Отобразить битовые образы в центре окна в виде квадрата размером 6x6. Цвет каждого изображения должен быть задан случайно.

Листинг 1

```
#include "glew.h"
#include "glut.h"
#include <stdlib.h>

// Растровый образ размером 24x24
GLubyte image[72] = {
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x18, 0x00,
    0x00, 0x3c, 0x00,
    0x00, 0x7e, 0x00,
    0x00, 0xe7, 0x00,
    0x01, 0xc3, 0x80,
    0x03, 0xff, 0xc0,
    0x07, 0x00, 0xe0,
    0x0e, 0x00, 0x70,
    0x1c, 0x00, 0x38,
    0x38, 0x00, 0x1c,
    0x39, 0x81, 0x9c,
    0x1d, 0x81, 0xb8,
    0x0f, 0x81, 0xf0,
    0x07, 0x00, 0xe0,
    0x03, 0x81, 0xc0,
    0x01, 0xc3, 0x80,
    0x00, 0xe7, 0x00,
    0x00, 0x7e, 0x00,
    0x00, 0x3c, 0x00,
    0x00, 0x18, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00
};

int windowHeight = 512;
int windowHeight = 512;

// Функция выполняет всю необходимую инициализацию в контексте визуализации
void SetupRC() {
    // Черный фон
    glClearColor(0.0f, 0.0f, 0.0f, 0.0f);

    // Строки битового образа выровнены по одному байту
    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
}

// Устанавливается система координат, согласованная с координатами окна
```

```

void ChangeSize(int w, int h) {
    // Предотвращает деление на ноль, когда окно слишком маленькое
    // (нельзя сделать окно нулевой ширины)
    if (h == 0)
        h = 1;

    windowWidth = w;
    windowHeight = h;

    glViewport(0, 0, w, h);

    // Система координат обновляется перед модификацией
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Псевдокоординаты окна
    gluOrtho2D(0.0, (GLfloat)w, 0.0f, (GLfloat)h);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

// Вызывается для рисования сцены
void RenderScene(void) {
    int x, y;
    const int imageSize = 24;
    const int imagesPerSide = 6;
    const int squareSize = imageSize * imagesPerSide;
    const int startX = (windowWidth - squareSize) / 2;
    const int startY = (windowHeight - squareSize) / 2;

    // Очищаем окно текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT);

    // Цикл из 6 строк и столбцов
    for (y = 0; y < imagesPerSide; y++) {
        for (x = 0; x < imagesPerSide; x++) {
            // Цвет каждого изображения задается случайно
            glColor3f((GLfloat)rand() / RAND_MAX,
                      (GLfloat)rand() / RAND_MAX,
                      (GLfloat)rand() / RAND_MAX);

            // Устанавливается растровое положение данного образа
            glRasterPos2i(startX + x * imageSize, startY + y * imageSize);

            // Рисуется битовый образ размером 24x24
            glBitmap(imageSize, imageSize, 0.0, 0.0, 0.0, 0.0, image);
        }
    }
}

```

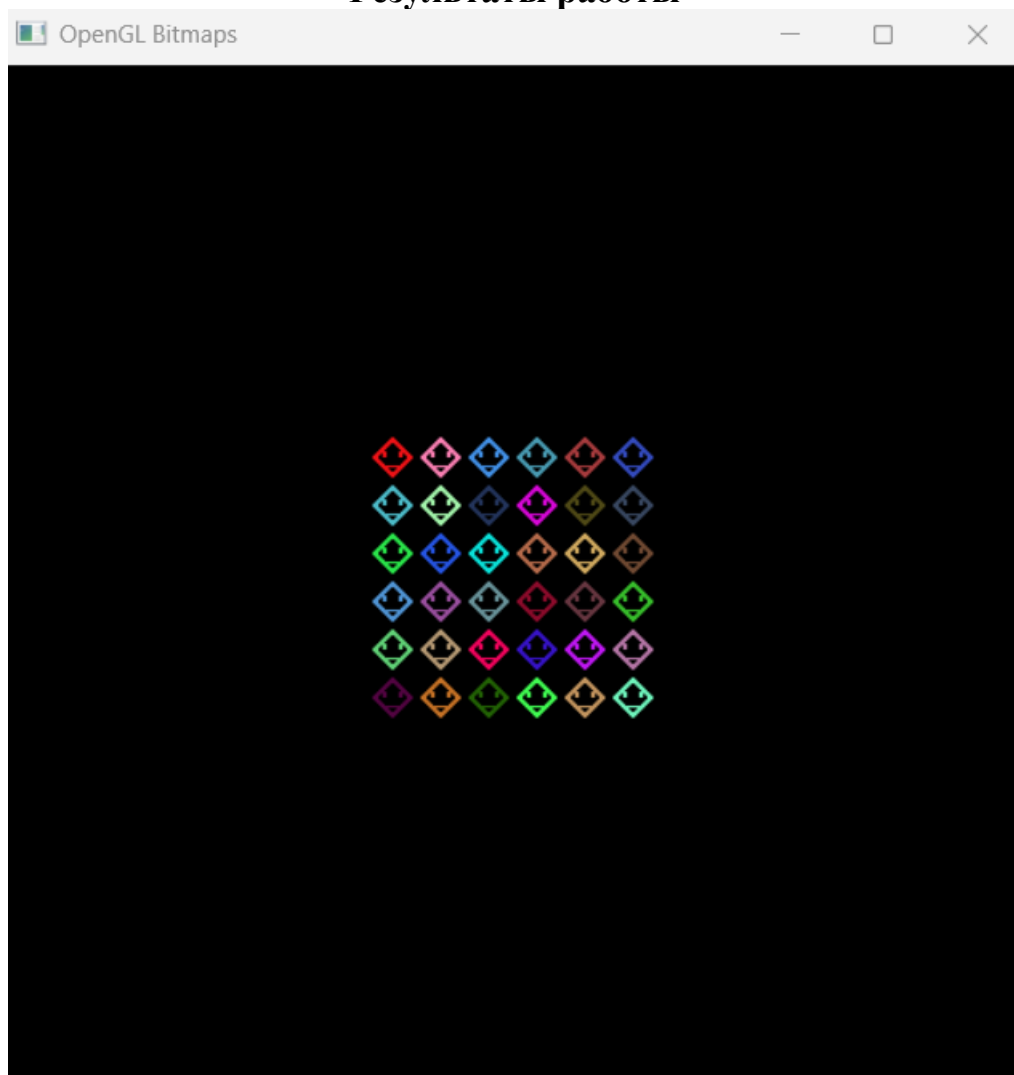
```

    // Переключает буферы
    glutSwapBuffers();
}

// Точка входа основной программы
int main(int argc, char *argv[]) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE);
    glutInitWindowSize(512, 512);
    glutCreateWindow("OpenGL Bitmaps");
    glutReshapeFunc(ChangeSize);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();
    return 0;
}

```

Результаты работы



Задание 2: Для собственного изображения формата tga, осуществить его вывод (4 раза). Исходное изображение и по компонентам RGB. Вывод начинается от правого верхнего до левого нижнего угла.

Листинг 2

```
#include "glew.h"
#include "glut.h"
#include <stdlib.h>

GLbyte *gltLoadTGA(const char *szFileName, GLint *iWidth, GLint *iHeight, GLint
*iComponents, GLenum *eFormat);

// Функция выполняет всю необходимую инициализацию в контексте визуализации
void SetupRC() {
    // Черный фон
    glClearColor(0.0f, 0.0f, 0.0f, 0.0f);
}

// Устанавливается система координат, согласованная с координатами окна
void ChangeSize(int w, int h) {
    if (h == 0)
        h = 1;

    glViewport(0, 0, w, h);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, (GLfloat)w, 0.0, (GLfloat)h);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void RenderScene(void) {
    GLubyte *pImage = NULL;
    GLint iWidth, iHeight, iComponents;
    GLint iViewport[4];
    GLenum eFormat;
    // Очищаем окно текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT);
    // Информация в файле Targa выравнена по одному байту
    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);

    // Загружает файл TGA и получает ширину, высоту, компоненты и формат
    pImage = (GLubyte *)gltLoadTGA("image.tga", &iWidth, &iHeight, &iComponents,
&eFormat);

    if (pImage != NULL) {
        glGetIntegerv(GL_VIEWPORT, iViewport);
```

```

// Рисуется исходное изображение в правом верхнем углу
glRasterPos2i(iViewport[2] - iWidth, iViewport[3] - iHeight);
glDrawPixels(iWidth, iHeight, eFormat, GL_UNSIGNED_BYTE, pImage);

// Рисуется красная компонента изображения
glPixelTransferf(GL_RED_SCALE, 1.0f);
glPixelTransferf(GL_GREEN_SCALE, 0.0f);
glPixelTransferf(GL_BLUE_SCALE, 0.0f);
glRasterPos2i(2 * (iViewport[2] - iWidth) / 3, 2 * (iViewport[3] -
iHeight) / 3);
glDrawPixels(iWidth, iHeight, eFormat, GL_UNSIGNED_BYTE, pImage);

// Рисуется зеленая компонента изображения
glPixelTransferf(GL_RED_SCALE, 0.0f);
glPixelTransferf(GL_GREEN_SCALE, 1.0f);
glPixelTransferf(GL_BLUE_SCALE, 0.0f);
glRasterPos2i((iViewport[2] - iWidth) / 3, (iViewport[3] - iHeight) / 3);
glDrawPixels(iWidth, iHeight, eFormat, GL_UNSIGNED_BYTE, pImage);

// Рисуется синяя компонента изображения в левом нижнем углу
glPixelTransferf(GL_RED_SCALE, 0.0f);
glPixelTransferf(GL_GREEN_SCALE, 0.0f);
glPixelTransferf(GL_BLUE_SCALE, 1.0f);
glRasterPos2i(0, 0);
glDrawPixels(iWidth, iHeight, eFormat, GL_UNSIGNED_BYTE, pImage);

// Возвращаются исходные значения цветовых компонент
glPixelTransferf(GL_RED_SCALE, 1.0f);
glPixelTransferf(GL_GREEN_SCALE, 1.0f);
glPixelTransferf(GL_BLUE_SCALE, 1.0f);
}

// Данные изображения уже не нужны
free(pImage);
// Переключает буферы
glutSwapBuffers();
}

// Точка входа основной программы
int main(int argc, char *argv[]) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE);
    glutInitWindowSize(800, 600);
    glutCreateWindow("OpenGL TGA Image");
    glutReshapeFunc(ChangeSize);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();
    return 0;
}

```

Листинг 3

```
#include "glew.h"
#include "glut.h"
#include <stdio.h>
#include <stdlib.h>

// Определение заголовка Targa.
#pragma pack(1)
typedef struct {
    GLbyte identSize; // Размер поля ID, следующего за заголовком (0)
    GLbyte colorMapType; // 0 = нет цветовой карты, 1 = есть цветовой карта
    GLbyte imageType; // 0 = нет, 1 = индексированное, 2 = RGB, 3 = серое, +8 =
RLE
    unsigned short colorMapStart; // Первая позиция цветовой карты
    unsigned short colorMapLength; // Количество цветов
    unsigned char colorMapBits; // Количество битов
    unsigned short xstart; // Начальная координата X
    unsigned short ystart; // Начальная координата Y
    unsigned short width; // Ширина в пикселях
    unsigned short height; // Высота в пикселях
    GLbyte bits; // Количество битов на пиксель (8, 16, 24 или 32)
    GLbyte descriptor; // Дескриптор изображения
} TGAHEADER;
#pragma pack(8)

// Распределяет память и загружает данные файла Targa. Возвращает указатель
// на новый буфер, высоту и ширину текстуры, а также формат данных OpenGL.
// После завершения работы буфер необходимо освободить функцией free().
// Поддерживаются простые файлы Targa с 8-, 24- или 32-битовым цветом,
// без палитры и группового кодирования.
GLbyte *gltLoadTGA(const char *szFileName, GLint *iWidth, GLint *iHeight, GLint
*iComponents,
                    GLenum *eFormat) {
    FILE *pFile; // Указатель на файл
    TGAHEADER tgaHeader; // Заголовок файла TGA
    unsigned long lImageSize; // Размер изображения в байтах
    short sDepth; // Размер пикселя
    GLbyte *pBits = NULL; // Указатель на данные изображения
    // Значения по умолчанию на случай ошибки.
    *iWidth = 0;
    *iHeight = 0;
    *eFormat = GL_BGR_EXT;
    *iComponents = GL_RGB8;
    pFile = fopen(szFileName, "rb");
    if (pFile == NULL)
        return NULL;
    // Считываем двоичный заголовок.
    fread(&tgaHeader, 18 /* sizeof(TGAHEADER) */, 1, pFile);
```



```

    // Изменяем порядок байтов на платформах с обратным порядком.
#ifdef __APPLE__
    BYTE_SWAP(tgaHeader.colorMapStart);
    BYTE_SWAP(tgaHeader.colorMapLength);
    BYTE_SWAP(tgaHeader.xstart);
    BYTE_SWAP(tgaHeader.ystart);
    BYTE_SWAP(tgaHeader.width);
    BYTE_SWAP(tgaHeader.height);
#endif
    *iWidth = tgaHeader.width;
    *iHeight = tgaHeader.height;
    sDepth = tgaHeader.bits / 8;
    // Поддерживаются только 8-, 24- и 32-битовые файлы Targa.
    if (tgaHeader.bits != 8 && tgaHeader.bits != 24 && tgaHeader.bits != 32) {
        fclose(pFile);
        return NULL;
    }
    lImageSize = tgaHeader.width * tgaHeader.height * sDepth;
    pBits = (GLbyte *)malloc(lImageSize * sizeof(GLbyte));
    if (pBits == NULL) {
        fclose(pFile);
        return NULL;
    }

    // Ошибка здесь может означать неподдерживаемое групповое кодирование
    // или другой формат, который функция не умеет распознавать.
    if (fread(pBits, lImageSize, 1, pFile) != 1) {
        free(pBits);
        fclose(pFile);
        return NULL;
    }

    // Устанавливаем формат, ожидаемый OpenGL.
    switch (sDepth) {
    case 3:
        *eFormat = GL_BGR_EXT;
        *iComponents = GL_RGB8;
        break;
    case 4:
        *eFormat = GL_BGRA_EXT;
        *iComponents = GL_RGBA8;
        break;
    case 1:
        *eFormat = GL_LUMINANCE;
        *iComponents = GL_LUMINANCE8;
        break;
    }

    fclose(pFile);
    return pBits;
}

```

Листинг 4

```
// Записывает текущее поле просмотра в файл Targa. Перед вызовом этой функции
// при двойной буферизации следует вызвать SwapBuffers, а при одинарной –
glFinish.
// Возвращает 0 при ошибке и 1 при успешном завершении.
GLint gltWriteTGA(const char *szFileName) {
    FILE *pFile;                // Указатель на файл
    TGAHEADER tgaHeader;        // Заголовок файла TGA
    unsigned long lImageSize;    // Размер изображения в байтах
    GLbyte *pBits = NULL;       // Указатель на данные изображения
    GLint iViewport[4];         // Размер поля просмотра в пикселях
    GLenum lastBuffer;          // Предыдущее значение буфера чтения

    glGetIntegerv(GL_VIEWPORT, iViewport);

    // Файлы Targa плотно упакованы и содержат по три байта на пиксель.
    lImageSize = iViewport[2] * 3 * iViewport[3];

    pBits = (GLbyte *)malloc(lImageSize);
    if (pBits == NULL)
        return 0;

    // Настраиваем плотную упаковку данных из буфера цвета.
    glPixelStorei(GL_PACK_ALIGNMENT, 1);
    glPixelStorei(GL_PACK_ROW_LENGTH, 0);
    glPixelStorei(GL_PACK_SKIP_ROWS, 0);
    glPixelStorei(GL_PACK_SKIP_PIXELS, 0);

    // Сохраняем текущий буфер чтения, читаем передний буфер и восстанавливаем
    // настройку.
    glGetIntegerv(GL_READ_BUFFER, (GLint *)&lastBuffer);
    glReadBuffer(GL_FRONT);
    glReadPixels(0, 0, iViewport[2], iViewport[3], GL_BGR_EXT, GL_UNSIGNED_BYTE,
    pBits);
    glReadBuffer(lastBuffer);

    // Инициализируем заголовок файла Targa.
    tgaHeader.identsize = 0;
    tgaHeader.colorMapType = 0;
    tgaHeader.imageType = 2;
    tgaHeader.colorMapStart = 0;
    tgaHeader.colorMapLength = 0;
    tgaHeader.colorMapBits = 0;
    tgaHeader.xstart = 0;
    tgaHeader.ystart = 0;
    tgaHeader.width = iViewport[2];
    tgaHeader.height = iViewport[3];
    tgaHeader.bits = 24;
    tgaHeader.descriptor = 0;
```

```

#ifdef __APPLE__
    BYTE_SWAP(tgaHeader.colorMapStart);
    BYTE_SWAP(tgaHeader.colorMapLength);
    BYTE_SWAP(tgaHeader.xstart);
    BYTE_SWAP(tgaHeader.ystart);
    BYTE_SWAP(tgaHeader.width);
    BYTE_SWAP(tgaHeader.height);
#endif

    pFile = fopen(szFileName, "wb");
    if (pFile == NULL) {
        free(pBits);
        return 0;
    }

    fwrite(&tgaHeader, sizeof(TGAHEADER), 1, pFile);
    fwrite(pBits, lImageSize, 1, pFile);
    free(pBits);
    fclose(pFile);
    return 1;
}

```

Результаты работы



Задание 3: Для собственного изображения формата tga выполнить увеличение по оси x в два раза, по оси y в 0.7 раза. Отобразить инвертированное изображение.

Листинг 5

```
#include "glew.h"
#include "glut.h"
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

// Глобальные переменные модуля для хранения исходных данных изображения
static GLbyte *pImage = NULL;
static GLint iWidth, iHeight, iComponents;
static GLenum eFormat;

// Глобальная переменная для хранения режима рисования
static GLint iRenderMode = 1;
// Определение заголовка targa
#pragma pack(1)
typedef struct {
    GLbyte identsize; // Размер поля ID, следующего за заголовком (0)
    GLbyte colorMapType; // 0 = нет цветовой карты, 1 = есть цветовой карта
    GLbyte imageType; // 0 = нет, 1 = индексированное, 2 = RGB, 3 = серое, +8 =
RLE
    unsigned short colorMapStart; // Первая позиция цветовой карты
    unsigned short colorMapLength; // Количество цветов
    unsigned char colorMapBits; // Количество битов
    unsigned short xstart; // Начальная координата X
    unsigned short ystart; // Начальная координата Y
    unsigned short width; // Ширина в пикселях
    unsigned short height; // Высота в пикселях
    GLbyte bits; // Количество битов на пиксель
    GLbyte descriptor; // Дескриптор изображения
} TGAHEADER;

#pragma pack(8)

// Эта функция определена в листинге 3
GLbyte *glLoadTGA(const char *szFileName, GLint *iWidth, GLint *iHeight, GLint
*iComponents,
                GLenum *eFormat);

// Выполняет необходимую инициализацию в контексте визуализации
void SetupRC(void) {
    // Черный фон
    glClearColor(0.0f, 0.0f, 0.0f, 0.0f);

    // Загружаем изображение
    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
```

```

    pImage = (GLubyte *)glLoadTGA("image.tga", &iWidth, &iHeight, &iComponents,
&eFormat);
}

void ShutdownRC(void) {
    // Освобождаем исходные данные изображения
    free(pImage);
}

// Должным образом обновляем флаги в ответ на выбор позиции из меню
void ProcessMenu(int value) {
    // Меняем индекс режима визуализации на индекс,
    // соответствующий позиции меню
    iRenderMode = value;

    // Активируем перерисовывание изображения
    glutPostRedisplay();
}

// Вызывается для рисования сцены
void RenderScene(void) {
    GLfloat invertMap[256];
    GLint i;

    // Очищаем окно текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT);

    // Текущее растровое положение соответствует левому нижнему углу окна
    glRasterPos2i(0, 0);

    // В зависимости от индекса режима визуализации выполняются
    // необходимые операции с изображением
    switch (iRenderMode) {
    case 2: // Масштабирование и инверсия цветов согласно варианту 3
        glPixelZoom(2.0f, 0.7f);

        for (i = 0; i < 256; i++)
            invertMap[i] = 1.0f - (GLfloat)i / 255.0f;

        glPixelMapfv(GL_PIXEL_MAP_R_TO_R, 256, invertMap);
        glPixelMapfv(GL_PIXEL_MAP_G_TO_G, 256, invertMap);
        glPixelMapfv(GL_PIXEL_MAP_B_TO_B, 256, invertMap);
        glPixelTransferi(GL_MAP_COLOR, GL_TRUE);
        break;

    case 1: // Просто копия старого изображения
    default:
        break;
    }

    // Рисуются пиксели

```

```

    if (pImage != NULL)
        glDrawPixels(iWidth, iHeight, eFormat, GL_UNSIGNED_BYTE, pImage);

    // Обновление всего до настроек по умолчанию
    glPixelTransferi(GL_MAP_COLOR, GL_FALSE);
    glPixelTransferf(GL_RED_SCALE, 1.0f);
    glPixelTransferf(GL_GREEN_SCALE, 1.0f);
    glPixelTransferf(GL_BLUE_SCALE, 1.0f);
    glPixelZoom(1.0f, 1.0f); // Без увеличения пикселей

    // Переключает буферы
    glutSwapBuffers();
}

void ChangeSize(int w, int h) {
    // Предотвращает деление на нуль, когда окно слишком маленькое
    // (нельзя сделать окно нулевой ширины)
    if (h == 0)
        h = 1;

    glViewport(0, 0, w, h);

    // Система координат обновляется перед модификацией
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Устанавливается объем отсечения
    gluOrtho2D(0.0f, (GLfloat)w, 0.0, (GLfloat)h);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

// Точка входа основной программы

int main(int argc, char *argv[]) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB | GL_DOUBLE);
    glutInitWindowSize(400, 300);
    glutCreateWindow("OpenGL Image Operations");
    glutReshapeFunc(ChangeSize);
    glutDisplayFunc(RenderScene);

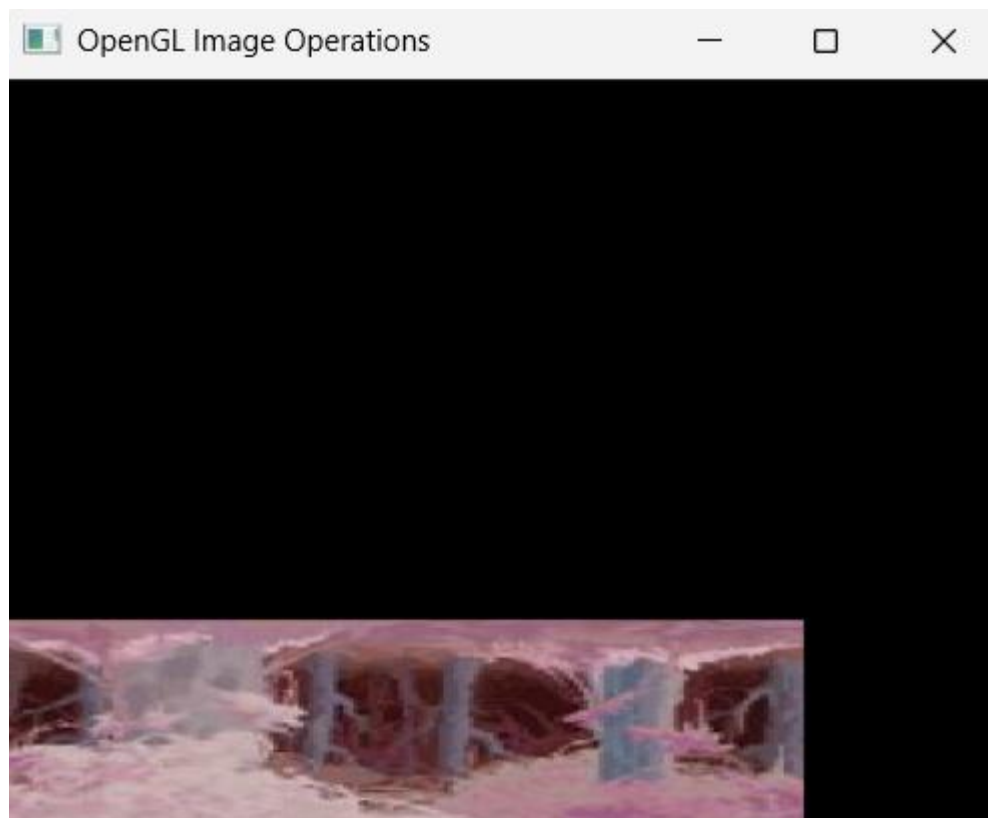
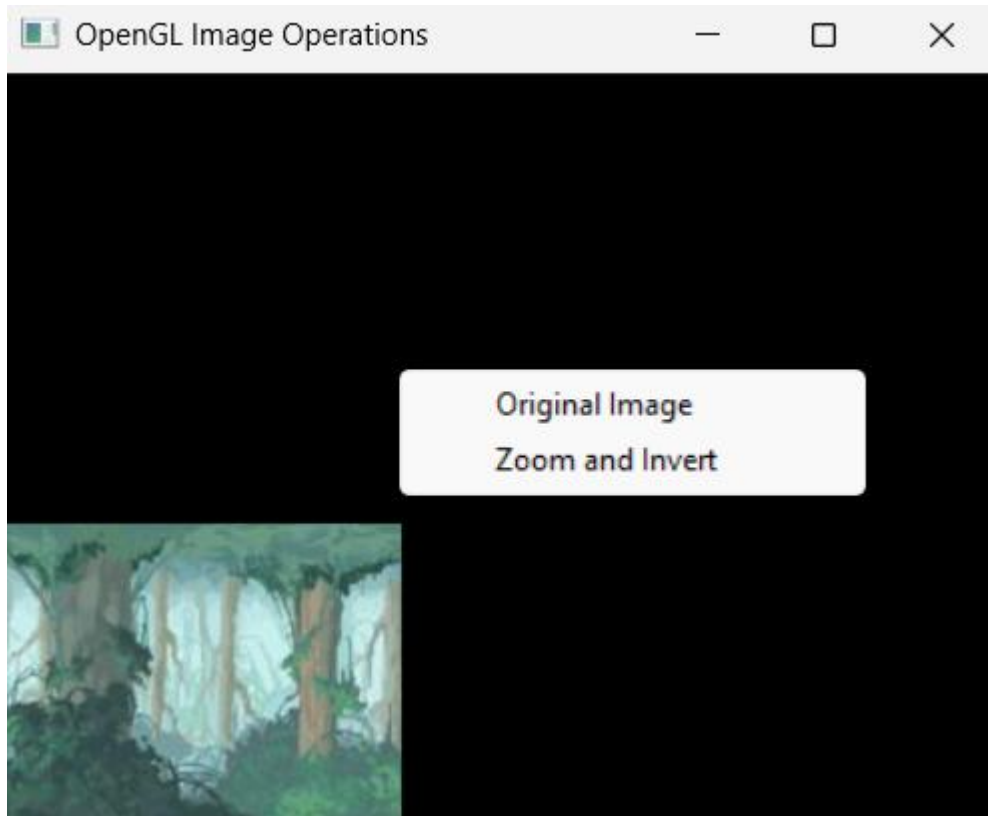
    // Создается меню и добавляются опции выбора
    glutCreateMenu(ProcessMenu);
    glutAddMenuEntry("Original Image", 1);
    glutAddMenuEntry("Zoom and Invert", 2);
    glutAttachMenu(GLUT_RIGHT_BUTTON);

    SetupRC(); // Настройка

```

```
}  
    glutMainLoop(); // Основной программный цикл  
    ShutdownRC(); // Выключение  
    return 0;  
}
```

Результаты работы



Вывод: в ходе работы были получены практические навыки по работе с растровыми изображениями средствами OpenGL, а также простейшим преобразованиям цветовых и пространственных характеристик.